

Benchmarking and Performance Analysis of LLM Inferencing Systems

Saksham Rathi

Systems for LLMs
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

Contents

1. Motivation
2. Motivation
3. Initial Experiments
4. Key Idea
5. Chunked Hash Tree
6. RL Decision Maker
7. Results

Motivation

- Modern LLM inference is often **memory-bandwidth bound** during decode
- KV-cache accesses dominate performance for long-context workloads
- Existing systems primarily optimize **HBM capacity utilization**
- However, decode efficiency also depends on **temporal and spatial locality**
- Current schedulers do not explicitly account for **cache behavior or DRAM bandwidth utilization**

Core Scheduling Problem

- Multiple requests often share long token prefixes
- During decode, batches issue simultaneous KV-cache memory requests
- Heterogeneous batches reduce:
  - Cache hit rates
  - DRAM access efficiency
  - Effective bandwidth utilization
- Fully homogeneous batches improve locality but may hurt system-level efficiency

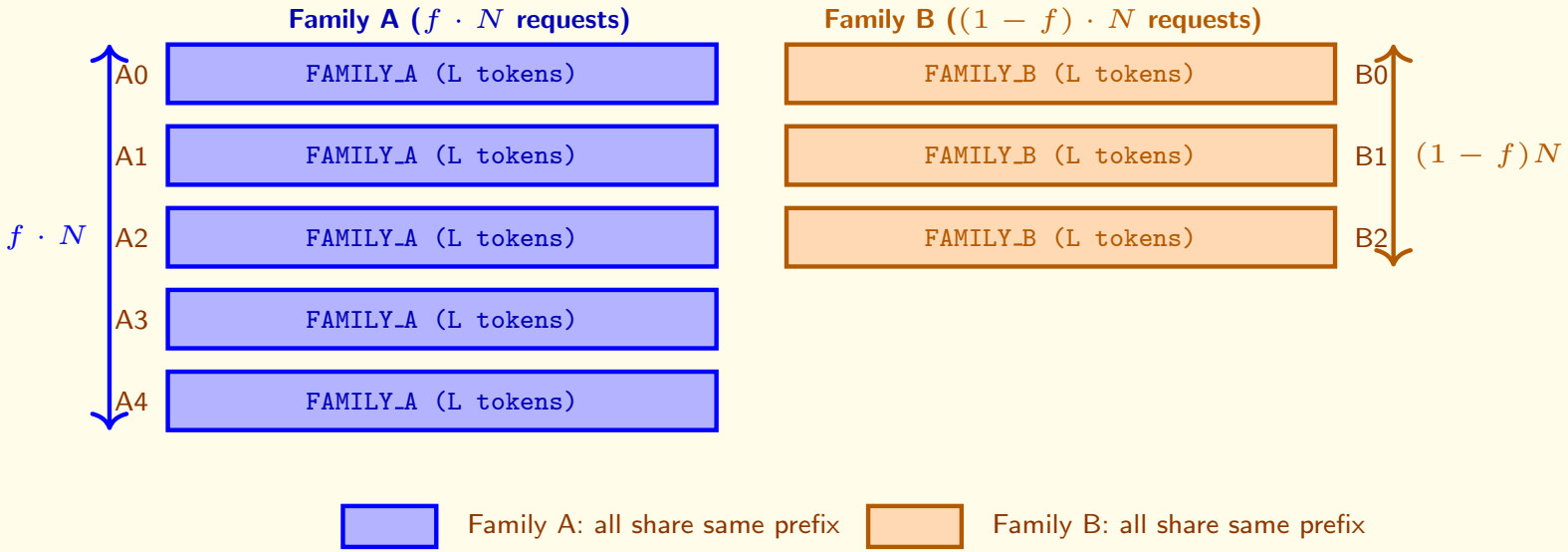
Goal

Construct batches that balance GPU utilization and prefix locality



Initial Experiments

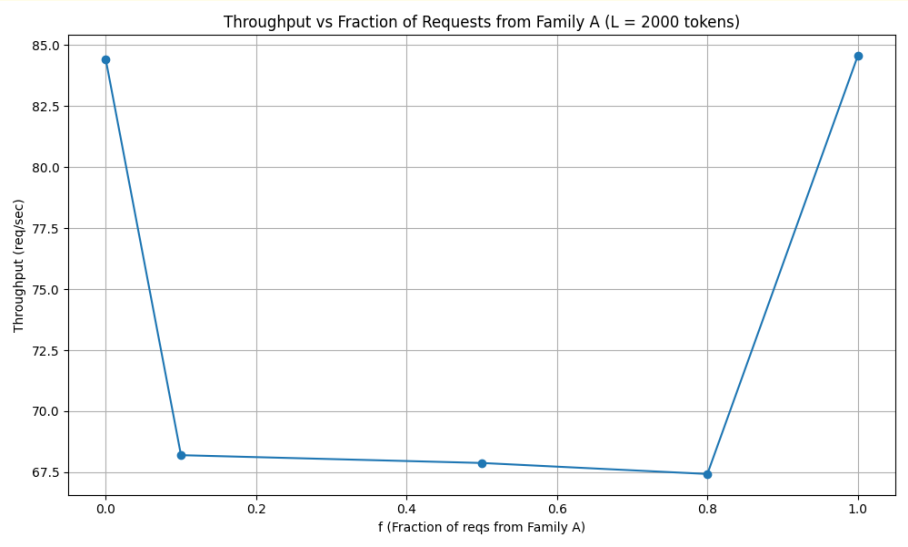
Two-family batch mixture



- Synthetic workload with **two distinct long prefixes** (Family A and B)
- All requests are in the **decode phase** (prefill already completed)
- Each prefix has length L tokens
- A fraction f of the batch belongs to Family A, and $(1-f)$ to Family B
- We vary f to control the **degree of batch heterogeneity**

Systems for LLMs
Saksham
Motivation
Motivation
Initial Experiments
Key Idea
Chunked Hash Tree
RL Decision Maker
Results

Two Family Batch Mixture ($L = 2000$ tokens)



Setup:

- Decode-only workload
- Prefix length $L = 2000$
- Fraction f controls batch heterogeneity

Observations:

- Throughput peaks at homogeneous batches ($f = 0$ or $f = 1$)
- Even slight heterogeneity causes sharp degradation

Key Takeaway

Batch composition strongly impacts decode throughput.

Two Family Mixture (L = 2000 tokens) - Mean DRAM Activity

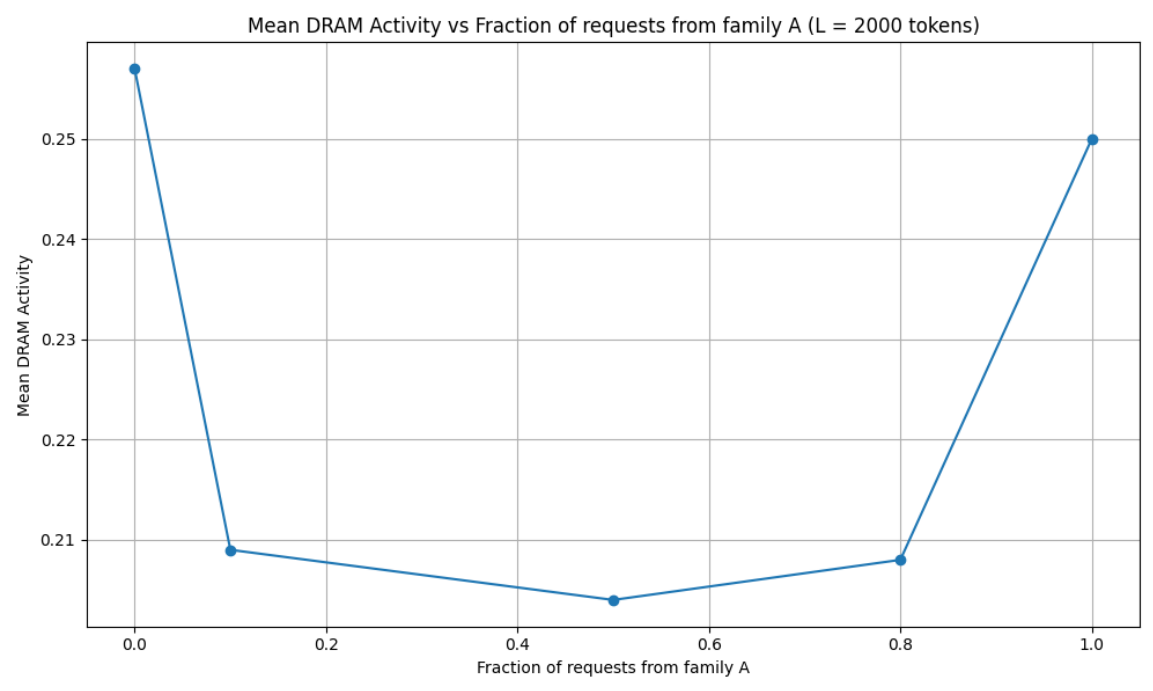


Figure: Mean DRAM Activity across fraction of requests from A (L = 2000 tokens)

- DRAM activity higher for homogeneous batches
- Sequential accesses → more spatial locality → higher prefetcher confidence → more requests issued → higher DRAM utilization

Systems for LLMs
Saksham
Motivation
Motivation
Initial Experiments
Key Idea
Chunked Hash Tree
RL Decision Maker
Results

Two Family Mixture (L = 2000 tokens) - Total DRAM Accesses

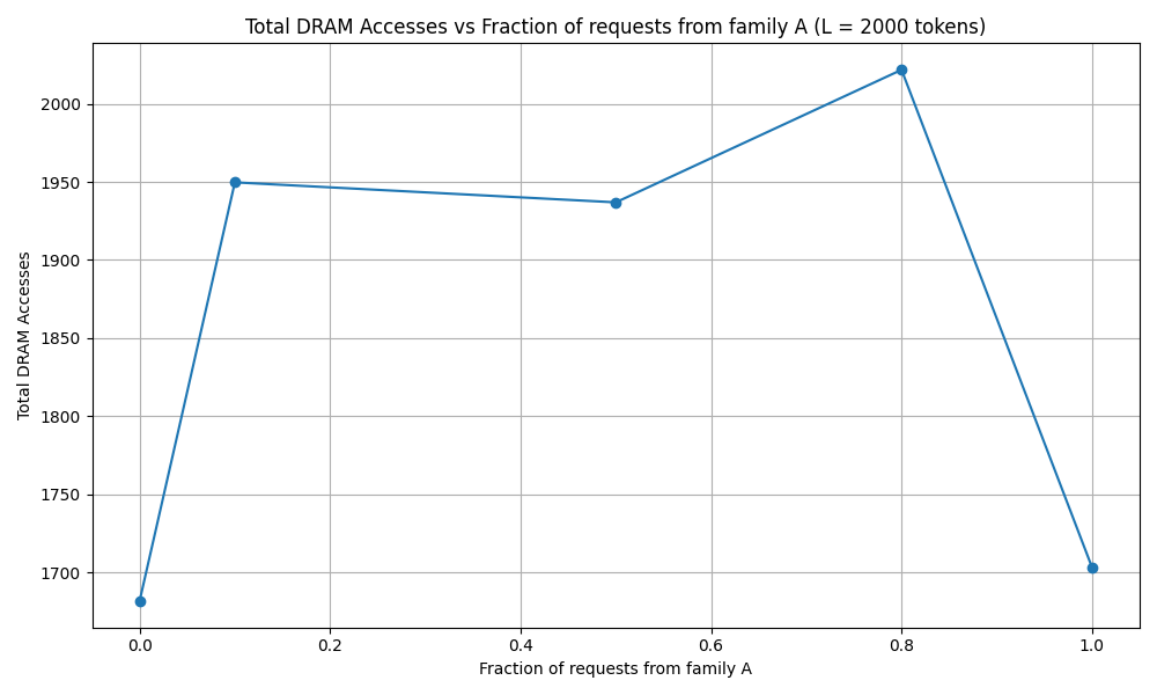
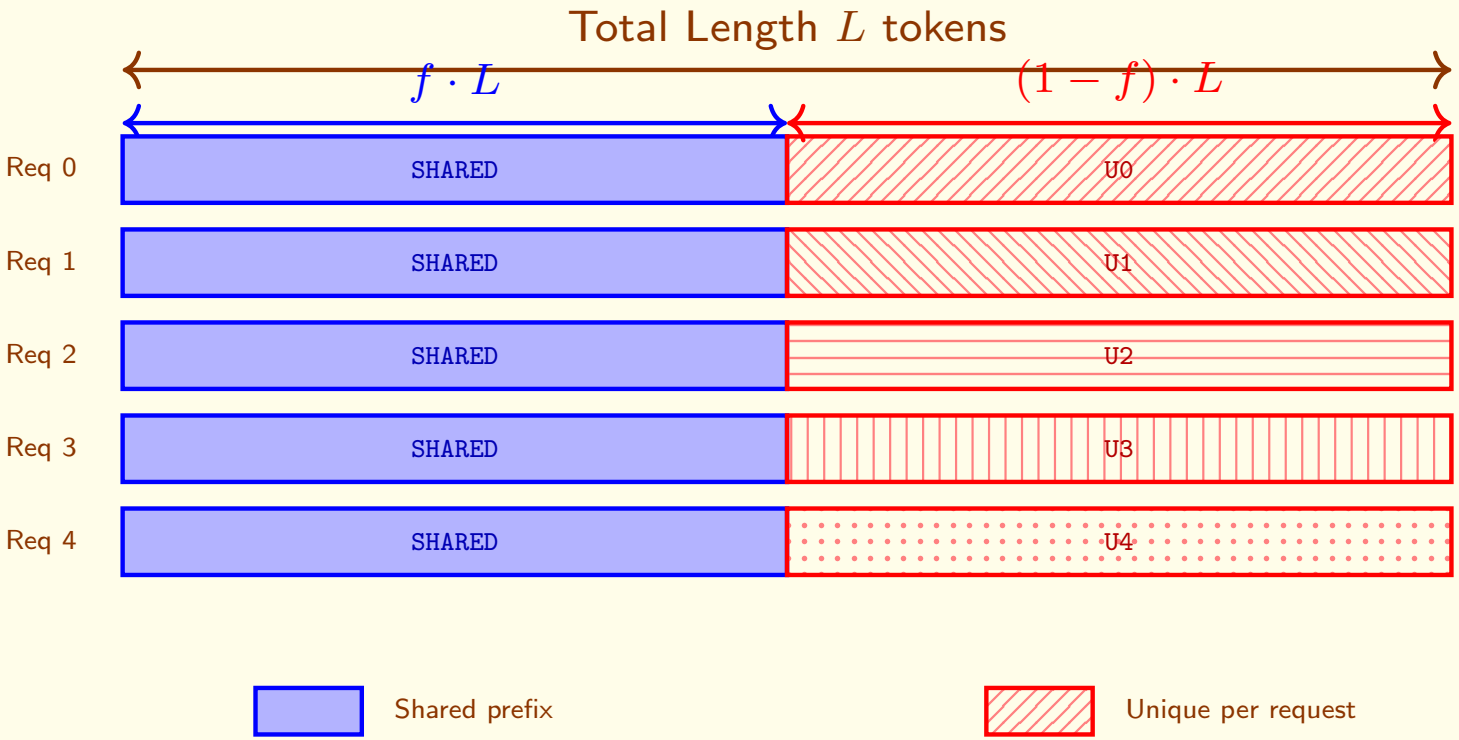


Figure: Total DRAM Accesses across fraction of requests from A (L = 2000 tokens)

- Total number of DRAM accesses lower for homogeneous batches
- Requests running in parallel → higher number of requests sharing the same block → better temporal locality → larger cache hit rates → less DRAM accesses

Systems for LLMs
Saksham
Motivation
Motivation
Initial Experiments
Key Idea
Chunked Hash Tree
RL Decision Maker
Results

Varying the amount of sharing



- Single-family workload: all requests originate from the same prefix
- Total sequence length fixed at L
- A fraction f of the prefix is shared across all requests
- Remaining $(1-f)L$ tokens are unique per request

Goal: Isolate the impact of prefix sharing on decode throughput.

Varying the amount of sharing (L = 500 tokens)

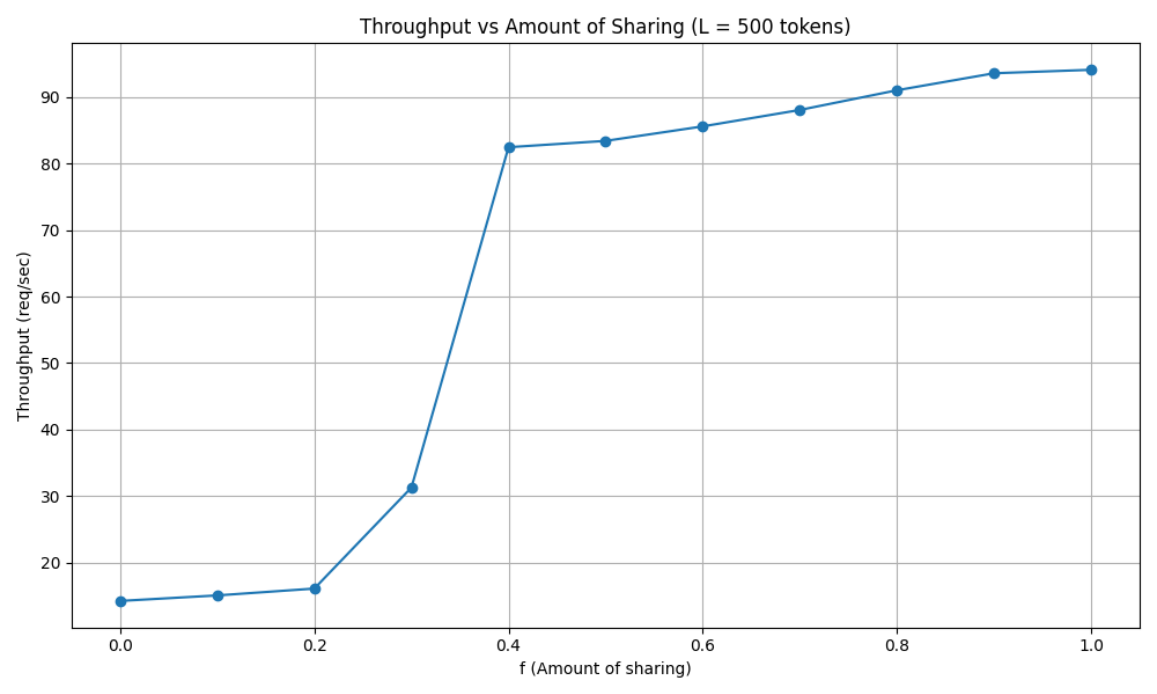


Figure: Results across varying L

- Decode-only workload, $L = 500$
- Low throughput at small f due to HBM KV Cache evictions
- Throughput increases steadily as shared prefix length grows
- Larger shared region $\rightarrow$ longer duration of spatial and temporal locality

Systems for LLMs
Saksham
Motivation
Motivation
Initial Experiments
Key Idea
Chunked Hash Tree
RL Decision Maker
Results

Key Conclusions from the Initial Experiments

- Decode-phase memory requests from batched requests are issued nearly **simultaneously**
- **Sequential** KV accesses improve effective DRAM bandwidth utilization
- Shared-prefix batches increase **temporal** and **spatial** locality
- Batches should be completely **homogeneous** to utilize these benefits fully
- **Greater prefix overlap** across active requests improves cache hit rates
- Merging requests from the same prefix together can also help in efficient KV cache eviction from the HBM $\rightarrow$ **avoiding recomputation**

Key Metric and Direction

Objective:

- Maximize concurrent GPU utilization
- Maximize shared-prefix locality duration

Metric:

(Batch Size, Shared Prefix Chunks (common to **all** active requests))

- Larger batch size $\rightarrow$ higher GPU utilization
- More shared chunks $\rightarrow$ sustained temporal and spatial locality
- Only chunks shared by **all** requests contribute to consistent reuse

Design Questions:

- How do we efficiently compute shared-prefix overlap at scale? $\rightarrow$ **Chunked Hash Tree**
- When should we stop adding requests to a batch? $\rightarrow$ **Reinforcement Learning (RL) Decision Maker**



Chunked Hash Tree

Why We Need a New Data Structure

- Scheduling requires computing:
  Shared Chunks Across Active Requests
- Naively comparing token prefixes is:
  - Too slow for long contexts (10K+ tokens)
  - Too expensive at high request rates
- Radix trees:
  - Require node splits
  - Expensive pointer traversal

Requirement

We need a faster overlap computation without explicit tree traversal.

Chunked Hash Tree



Core Idea

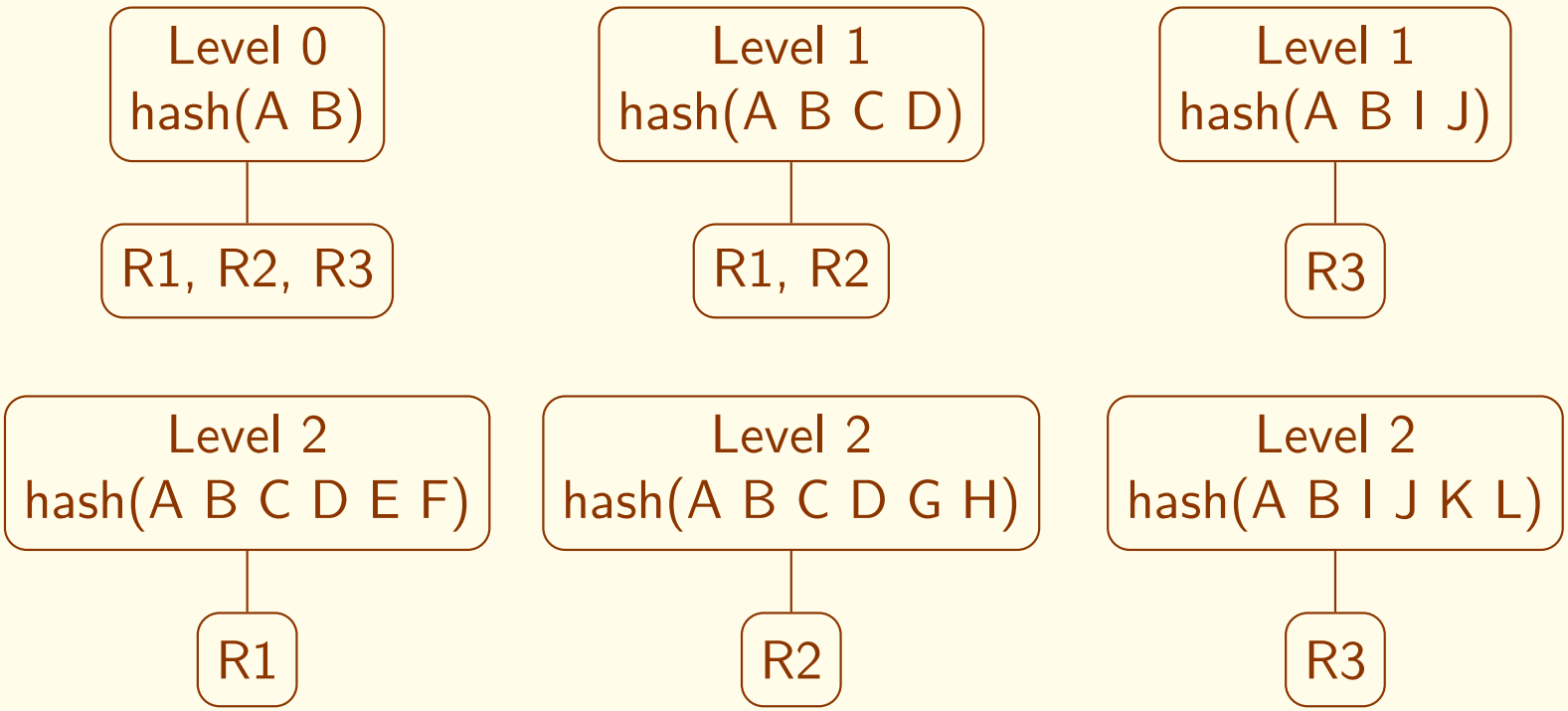
Convert prefix-overlap computation into hash-set intersection over chunked prefixes.

- Divide tokens into fixed-size chunks of size c
- For each request, compute cumulative prefix hashes: $h_l = hash(tokens[0 : l \cdot c])$
- Represent each request as a set of **(level, hash)** pairs
- Working Set:

$$W = \{(l, h_l) \text{ for all active requests}\}$$

- Shared chunks = levels where all active requests share the same hash
- Overlap computation reduces to counting shared hashes (no tree traversal, no node splitting)

Chunked Hash Tree - Insertion



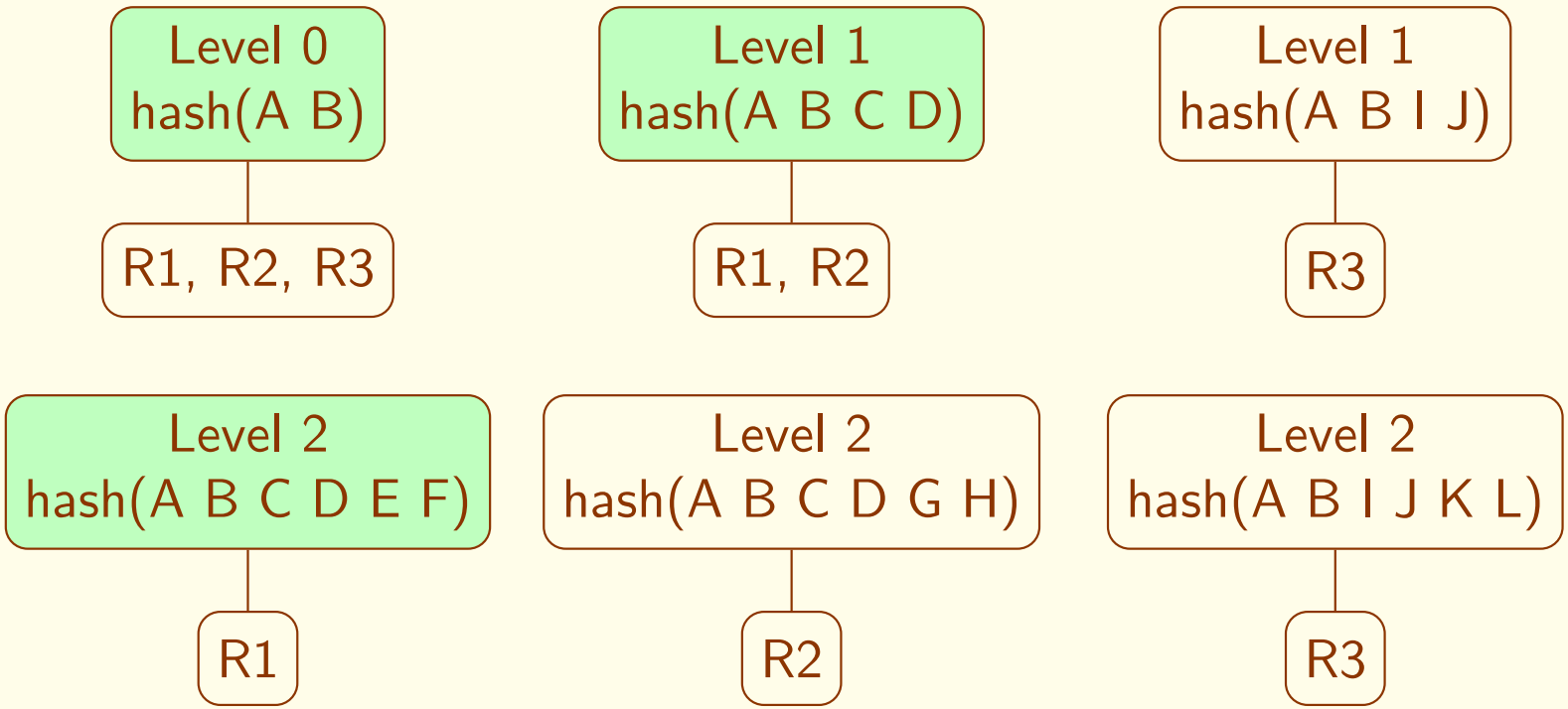
Each level corresponds to one chunk (size c tokens).

Nodes represent cumulative prefix hashes $h_l = hash(tokens[0 : l \cdot c])$.

- For each chunk, compute its cumulative hash
- Add request ID to corresponding (level, hash) bucket

Systems for LLMs
Saksham
Motivation
Motivation
Initial Experiments
Key Idea
Chunked Hash Tree
RL Decision Maker
Results

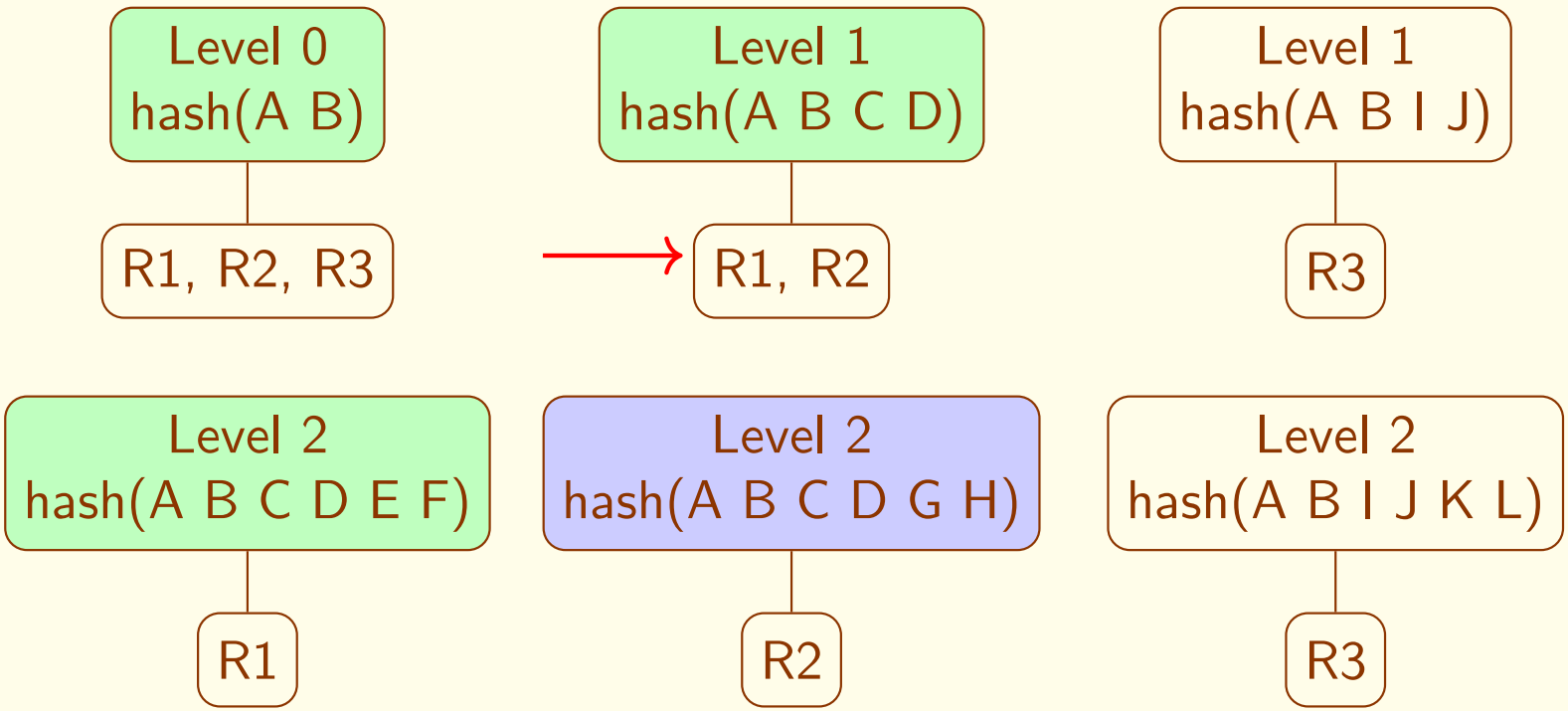
Chunked Hash Tree - Working Set



Working Set W = all (level, hash) pairs corresponding to currently active requests.

Shared chunks = levels where all active requests map to the same hash.

Chunked Hash Tree - Best Request Selection



For each candidate request:

- Count number of (level, hash) pairs shared with W

Select request that maximizes shared levels

Systems for LLMs
Saksham
Motivation
Motivation
Initial Experiments
Key Idea
Chunked Hash Tree
RL Decision Maker
Results



RL Decision Maker

Formulation

Tradeoff

Add more requests to increase GPU utilization, but risk reducing shared-prefix locality.

State

- **B:** current batch size
- $\mathbf{C_B}$: shared chunks across current active batch
- $\mathbf{C_A}$: shared chunks if best candidate is added
- $\mathbf{R_W}$: number of requests waiting at the end of C_A

Formulation

Action

- **ADD:** pop and activate the best request from the chunked hash tree
- **SKIP:** break the batch construction loop, dispatch the batch to GPU

Reward

- Negative decode wall step time of the previous batch
- Delivered once per batch

Policies

- **Q Learning:** Learns when sacrificing locality now may improve long-term throughput across future batches.
- **Contextual Bandit:** Optimizes immediate throughput of the next batch only.
- **Heuristics:** Static thresholds on batch size or chunk drop.



Results

Throughput vs Req Rate (5 families) (Sequence Length: 3900 tokens)

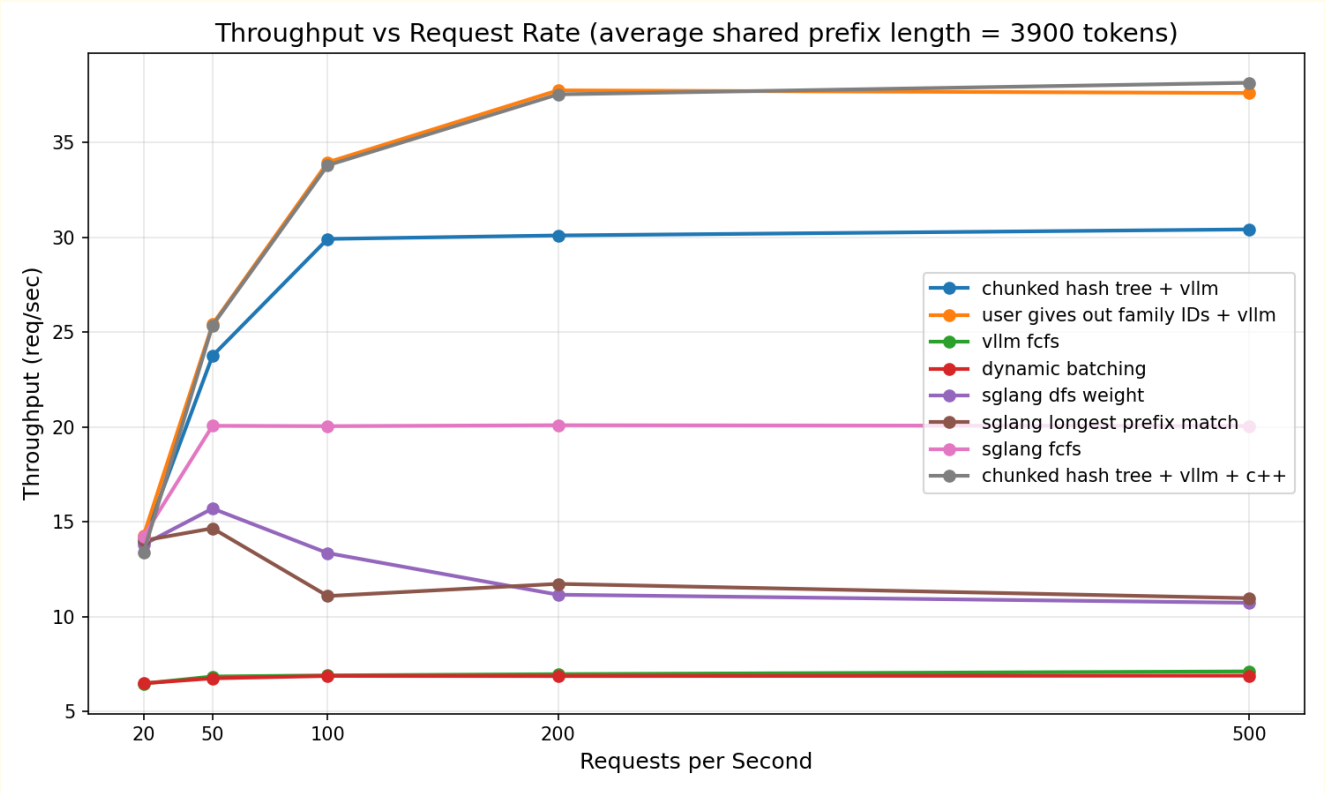


Figure: Throughput vs Request Rate (5 families)

Throughput vs Req Rate (5 families) (Sequence Length: 13000 tokens)

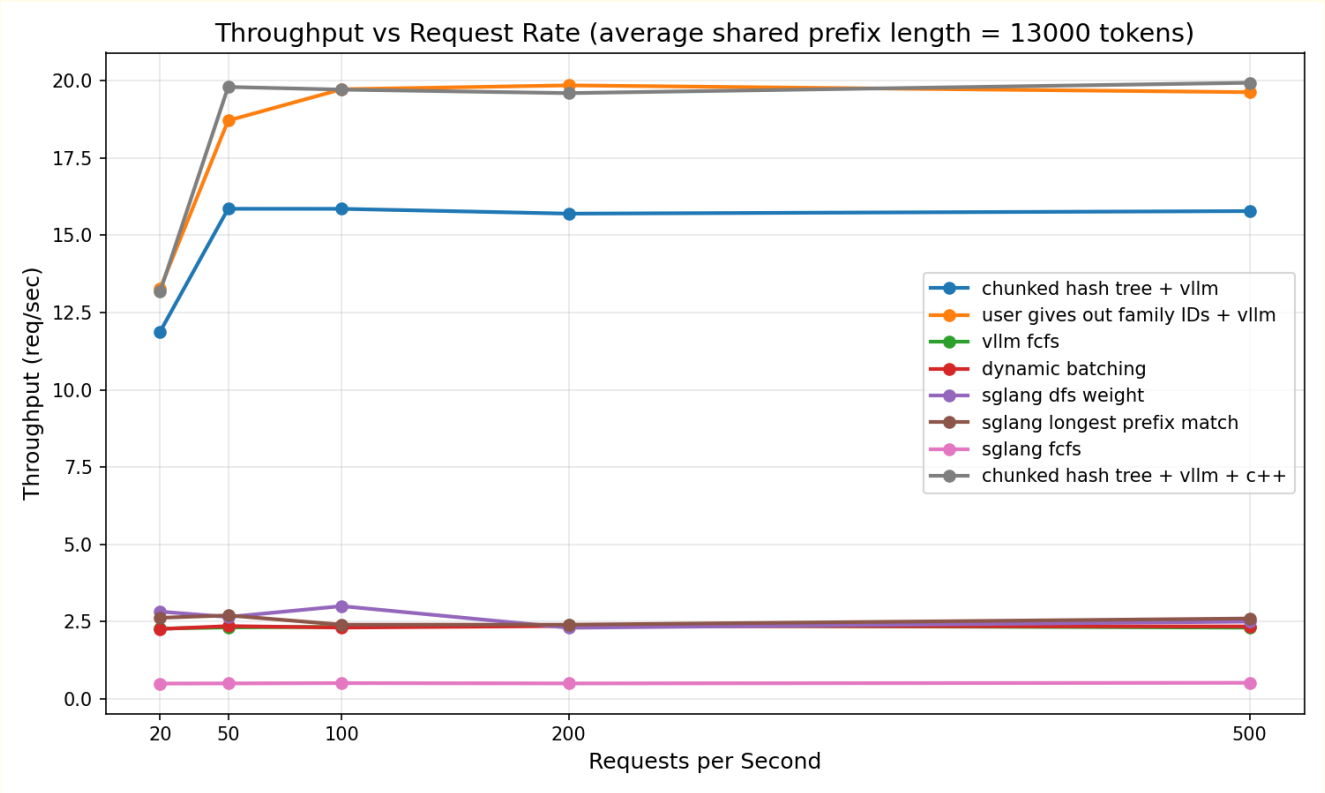


Figure: Throughput vs Request Rate (5 families)



Thank You